

601.433/633 Intro Algorithms

Recitation 2

Fall 2026

Lower bounds for comparison-based algorithms

Let A be an array of n distinct integers (not necessarily positive) where A is already sorted in ascending order. Our problem is to find an index i , $1 \leq i \leq n$, such that $A[i] = i$ or determine that no such i exists. Prove that any comparison-based algorithm for this problem must have $\Omega(\log n)$ worst-case running time.

In this context, “comparison-based” means that the algorithm is only allowed to access A through queries that compare an element of A to an arbitrary integer (since A is already sorted there is no need to compare two elements of A). These queries are either $<$, $=$, or $>$. So, for example, queries such as “is $A[i] < k$?” or “is $A[i] = j$?” are allowed.

Solution: First, note that if there is a valid index, it can be in any of the n locations of A . Each comparison we do rules out some locations while others are still possible, so we can think of any comparison based algorithm as a decision tree where each internal node is labeled with a particular query. Since each comparison is binary, there are two possible outcomes of any comparison, implying that the decision tree is a binary tree.

Consider input arrays which have at most a single valid index. No comparison-based search algorithm can stop when there are still multiple indices consistent with the comparisons it has made so far, and thus each leaf of this tree is a single index which must be uniquely identified by the outcomes corresponding to the path from it up to the root. Thus, our tree has at least n leaves. Given a tree, the number of comparisons it makes in the worst case is simply its height, i.e. the maximum over all leaves of their distance from the root. Since our decision tree has at least n leaves and is binary, it has height at least $\log n$. Hence any comparison-based algorithm has worst-case running time of at least $\Omega(\log n)$.

How large a universe can a constant number of linear-time passes handle?

Unless stated otherwise, we sort n integers. Throughout, assume that counting sort on n integers whose digits lie in $\{0, \dots, k-1\}$ takes $\Theta(n+k)$ time. Whenever counting sort is used as a subroutine for Least-Significant-Digit (LSD) radix sort, it is implemented stably.

- (a) Suppose an LSD radix-sort algorithm uses exactly d stable counting-sort passes. Each pass takes $O(n)$ time. Show that such an algorithm can represent at most $O(n^d)$ distinct integers.
- (b) Show conversely that if $U = O(n^d)$, then all integers in $\{0, \dots, U-1\}$ can be sorted in $O(dn)$ time using d radix passes.
- (c) Deduce that a constant number of $O(n)$ counting-sort passes can sort every integer in a polynomial-size universe $U = n^{O(1)}$, but cannot handle a superpolynomial universe with only a constant number of such passes.

Solution:

For part (a), an integer is represented by a d -tuple of digits. We are given that every pass takes $O(n)$ time. Suppose that in pass i , the digit takes values in a set of size k_i . Since counting sort takes time $\Theta(n + k_i)$, this implies that $k_i = O(n)$. Thus, the total number of distinct integers is at most

$$\prod_{i=1}^d k_i.$$

Since each $k_i = O(n)$, there is a constant C such that $k_i \leq Cn$ for all i , and therefore

$$\prod_{i=1}^d k_i \leq (Cn)^d = O(n^d)$$

for a fixed d .

For part (b), suppose

$$U \leq Cn^d$$

for some constant C . Choosing a radix (or base) $k = c'n$ for a constant c' such that $c'^d \geq C$ gives us $k^d \geq U$. Since $k = \Theta(n)$, each counting-sort pass costs

$$O(n + k) = O(n),$$

as representing the integers in the universe using base k implies that every digit of an integer takes values in $\{0, \dots, k-1\}$. Thus the total time is

$$O(dn).$$

For part (c), if the number of counting-sort passes d is constant, then the representable universe has size at most $n^{O(1)}$. Conversely, any polynomial universe $U = n^p$ with constant p can be handled using p base- n digits, hence a constant number of linear-time passes.

Therefore

constant many linear-time radix passes $\iff$ polynomial-size universe
--

up to constant factors in the radix.